

UNIVERSITÀ
DEGLI STUDI
DI MILANO

Workshop: Programming Lab

HUMAN-CENTERED ARTIFICIAL INTELLIGENCE

Lecture 4: Lists and Loops (WHILE- FOR loops)

Dr. Fatme Hachem

26/10/2026

Concept of iteration

What if we want to develop a program that **keeps asking** the user to enter item names and values **until** the user decides to stop?

→Iteration

Concept of iteration

Iteration is the process of repeatedly executing a set of statements.

$\Rightarrow$execute a block of code multiple times

In Python, there are two main types of iteration: Definite & Indefinite

Do we know beforehand the number of iterations required?

- Yes: then, it's definite (**for** loop)
- No, then it's indefinite (**while** loop) , in this case the iterations stop based on a condition

While loop

```
while condition:
    #code to be repeated
    #.....
    #update the condition items' value
    #so the block will not keep running
    #FOREVER
```

The loop will continue executing as long as the condition is True.

You need to make sure that the condition eventually becomes False; otherwise, the loop will run FOREVER.

What does this code do?

```
number = 1
# Start the while loop
while number <= 5:
    print(number)  # Print the current number
    number = number + 1     # Increment the number by 1 for the next iteration
```

Note: arithmetic operation over variables

What is the output of the following code:

```
number= 2

number +=1
print(number)

number -= 1
print(number)

number *= 4
print(number)

number /= 4
print(number)
```

Note: arithmetic operation over variables

What is the output of the following code:

```
number= 2

number +=1
print(number)

number -= 1
print(number)

number *= 4
print(number)

number /= 4
print(number)
```

Variable **+=** *value* ➔ **add** to the *variable* this *value*

Variable **-=** *value* ➔ **subtract** from the *variable* this *value*

Variable ***=** *value* ➔ **multiply** the *variable* by this *value*

Variable **/=** *value* ➔ **Divide** the *variable* by this *value*

Note: arithmetic operation over variables

What is the output of the following code:

```
number= 2

number +=1
print(number)

number -= 1
print(number)

number *= 4
print(number)

number /= 4
print(number)
```

Equivalent



```
number= 2

number = number +1
print(number)

number = number - 1
print(number)

number = number * 4
print(number)

number = number / 4
print(number)
```

The While loop: Example

Write a python code that takes from the user a **sentence**, and a number **n**, then prints out the **sentence n** times.

Output example:

```
Enter the sentence:  Hello!
How many times do you want to repeat this sentence 5
Hello!
Hello!
Hello!
Hello!
Hello!
```

The While loop: Example- a solution

Write a python code that takes from the user a **sentence**, and a number **n**, then prints out the **sentence n** times.

```
sentence= input('Enter the sentence: ')
n=0
try:
    n= int(input('How many times do you want to repeat this sentence'))
except ValueError:
    print('Only integers are allowed')

i=1
while i<=n:
    print(sentence)
    i+=1
```



What happens if we don't use the statement:
n=0
in the second line?

The While loop

How many times we're going to get "Hello" in each case?

```
i=1

while i<=3:
    print(i,'Hello')
    i+=1
```

```
i=0

while i<=3:
    print('Hello')
    i+=1
```

```
i=1

while i<3:
    print('Hello')
    i+=1
```

The While loop

How many times we're going to get "Hello" in each case?

```
i=1

while i<=3:
    print(i,'Hello')
    i+=1
```

```
i=0

while i<=3:
    print('Hello')
    i+=1
```

```
i=1

while i<3:
    print('Hello')
    i+=1
```

```
i= 1 : Hello
i= 2 : Hello
i= 3 : Hello
```

The While loop

How many times we're going to get "Hello" in each case?

```
i=1

while i<=3:
    print(i,'Hello')
    i+=1
```

```
i= 1 : Hello
i= 2 : Hello
i= 3 : Hello
```

```
i=0

while i<=3:
    print('Hello')
    i+=1
```

```
i= 0 : Hello
i= 1 : Hello
i= 2 : Hello
i= 3 : Hello
```

```
i=1

while i<3:
    print('Hello')
    i+=1
```

The While loop

How many times we're going to get "Hello" in each case?

```
i=1

while i<=3:
    print(i,'Hello')
    i+=1
```

```
i= 1 : Hello
i= 2 : Hello
i= 3 : Hello
```

```
i=0

while i<=3:
    print('Hello')
    i+=1
```

```
i= 0 : Hello
i= 1 : Hello
i= 2 : Hello
i= 3 : Hello
```

```
i=1

while i<3:
    print('Hello')
    i+=1
```

```
i= 1 : Hello
i= 2 : Hello
```

Exercise 1:

Write a Python program to calculate the factorial of a non-negative integer entered by the user.

The factorial of a number *n* is the product of all positive integers from 1 to *n*.

Example:

3!= 1*2*3

5! = 1*2*3*4*5

Exercise 1- a solution:

```
try:
    n=int(input('please enter an integer to calculate its factorial '))

    if n<0:
        print('your number should be positive')
    else:
        i=1
        factorial=1
        while i<=n:
            factorial*=i
            i+=1

        print('The factorial of ',n,  'is ',factorial)

except ValueError:
    print('You should enter a number only')
```

Exercise 1- a solution- enhanced (1):

What happens if we add this code to our solution:

```
should_ask_user=True

while should_ask_user:
    try:
        n=int(input('please enter an integer to calculate its factorial '))

        if n<0:
            print('your number should be positive')
        else:
            i=1
            factorial=1
            while i<=n:
                factorial*=i
                i+=1

            print('The factorial of ',n,  'is ',factorial)
            should_ask_user=False

    except ValueError:
        print('You should enter a number only')
```

Exercise 1- a solution- enhanced (1):

What happens if we add this code to our solution:

```
should_ask_user=True

while should_ask_user:
    try:
        n=int(input('please enter an integer to calculate its factorial '))

        if n<0:
            print('your number should be positive')
        else:
            i=1
            factorial=1
            while i<=n:
                factorial*=i
                i+=1

            print('The factorial of ',n,  'is ',factorial)
            should_ask_user=False

    except ValueError:
        print('You should enter a number only')
```

In this case, the result is returned with no problems, so we exit the big while loop

In these two cases: ***should_ask_user*** is still **True**, so the big while loop will iterate again.

Exercise 1- a solution- enhanced (2):

Now let's complete it a bit more:

```
continue_runing=True

while continue_runing:
    try:
        n=int(input('please enter an integer to calculate its factorial '))

        if n<0:
            print('your number should be positive')
        else:
            i=1
            factorial=1
            while i<=n:
                factorial*=i
                i+=1

            print('The factorial of ',n,  'is ',factorial)

            x=input('Continue? press Y if yes, any other character to exit: ')
            if x!='Y' and x!='y':
                continue_runing=False


    except ValueError:
        print('You should enter a number only')
```

Exercise 1- Question

What if we put the **try-except** outside the **while** loop?

```
try:
    continue_runing=True

    while continue_runing:

            n=int(input('please enter an integer to calculate its factorial '))

            if n<0:
                print('your number should be positive')
            else:
                i=1
                factorial=1
                while i<=n:
                    factorial*=i
                    i+=1

                print('The factorial of ',n,  'is ',factorial)

                x=input('Continue? press Y if yes, any other character to exit: ')
                if x!='Y' and x!='y':
                    continue_runing=False

except ValueError:
        print('You should enter a number only')
```

Data structure: List

What is a List?

A list is a built-in data type that can hold an ordered collection of items

```
my_grades=[25,28,30,18, 21]
my_courses=['math', 'sciences', 'languages']
```

The items are separated by a comma «**,**»

The list is defined in between square brackets **«[ ]»**

List: characteristics

Lists are **ordered** and **mutable**.

Ordered: means the items in a list have a definite order, and you can access them by their index.

Mutable: means you can change their content, by: adding, removing, or modifying existing items.

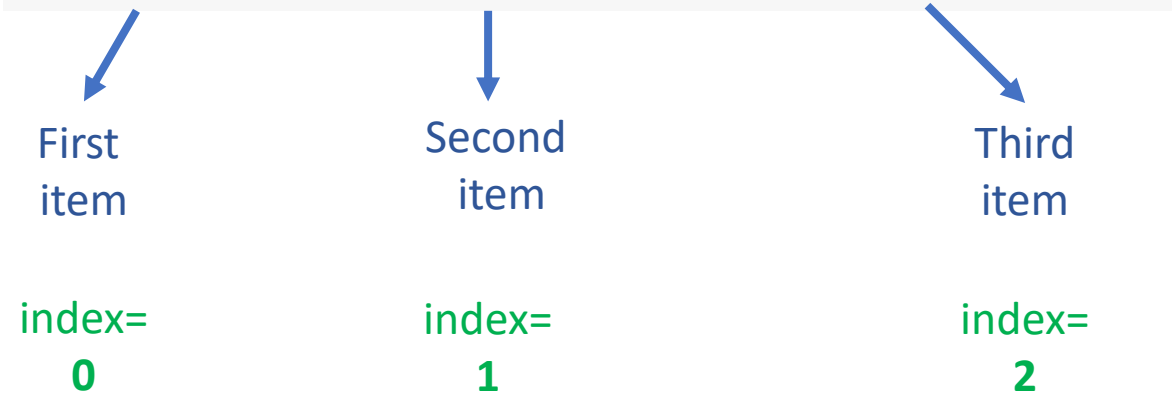
List: Index

Ordered: means the items in a list have a definite order, and you can access them by their **index**.

What is an index?

List of length=3

```
['math', 'sciences', 'languages']
```



⚠ Attention: the index starts with 0, not with 1!!

So, how to access elements in a list?

You can access elements in a list using indexing.

```
my_courses=['math', 'sciences', 'languages']


print(my_courses[0])
print(my_courses[1])
print(my_courses[2])
```

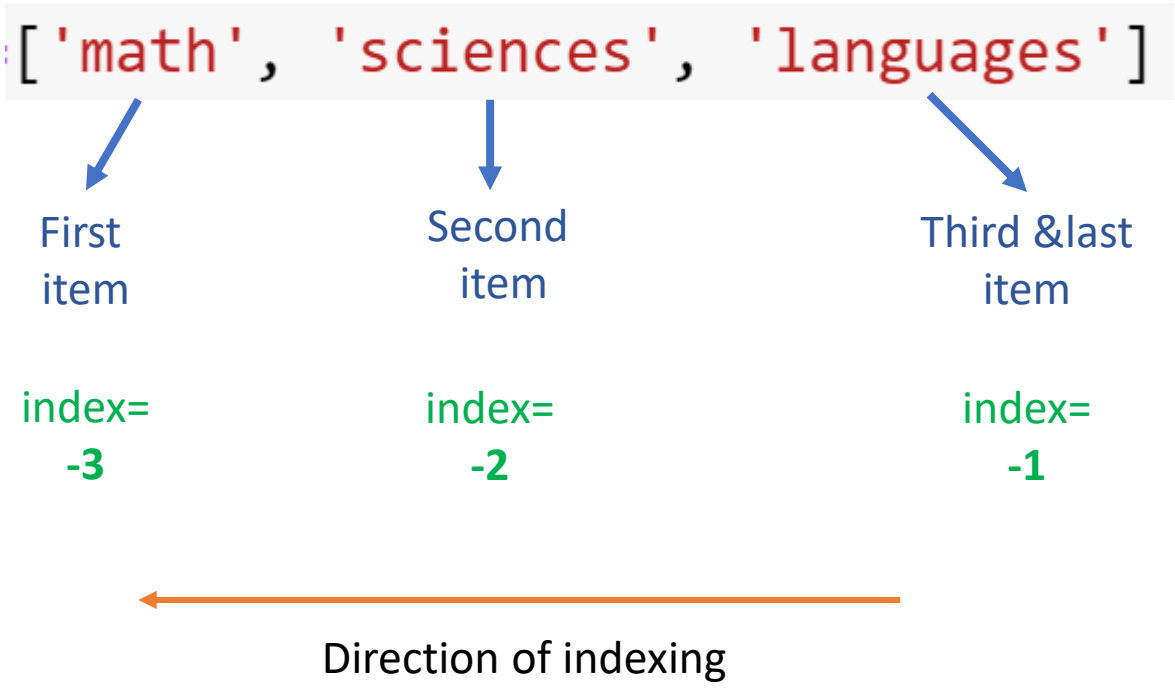
Output:

```
math
sciences
languages
```

Index can be backwards too! 😎

We can also use negative indexing, which starts from -1 for the last element in the list

List of length=3



So, how to access elements in a list using negative indexing?

```
my_courses=['math', 'sciences', 'languages']


print(my_courses[-1])
print(my_courses[-2])
print(my_courses[-3])
```

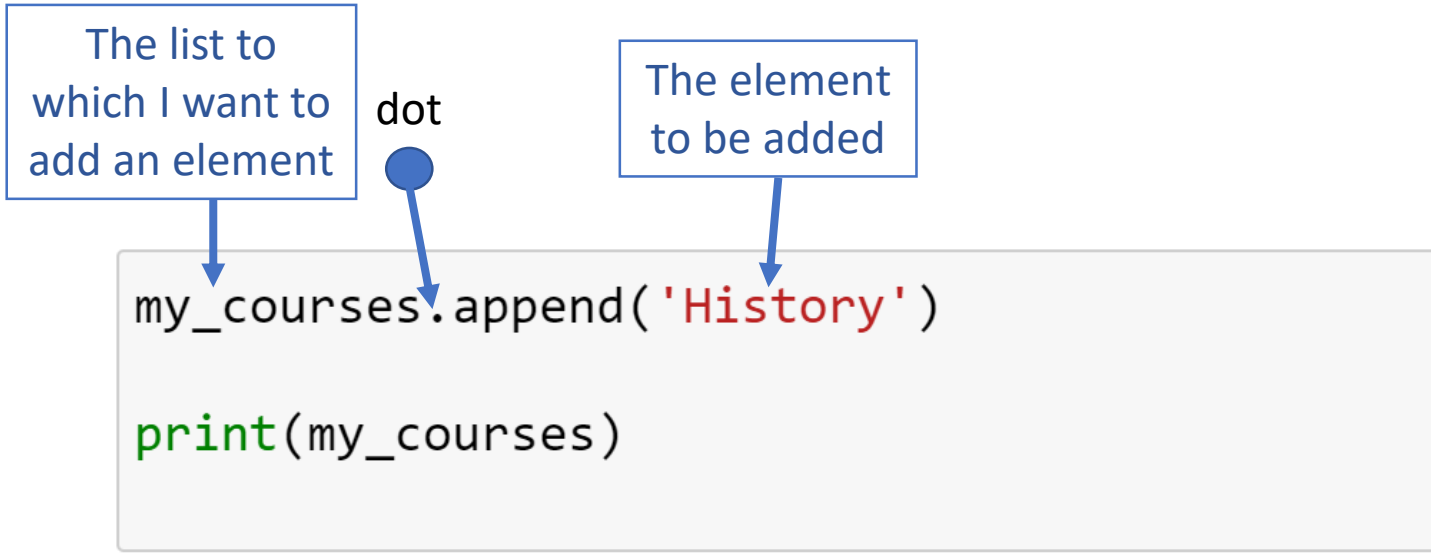
Output:

```
languages
sciences
math
```

Lists are mutable: add an element

Lists come with a variety of built-in methods that allow you to perform different operations, such as adding or removing elements, sorting…

How to add an element to a list?....Using “append” function



Lists are mutable: remove an element

How to remove an element from a list?....Using “remove” function

```
my_courses.remove('languages')

print(my_courses)
```

```
['math', 'sciences', 'History']
```

Lists are mutable: modifying an element

How to modify an element in a list?

```
my_courses[0]='mathematics'

print(my_courses)
```

```
['mathematics', 'sciences', 'History']
```

Length of a list

How to find the length of a list? …by using **len()**

```
my_courses= ['mathematics', 'sciences', 'History']

print(len(my_courses))

my_courses.remove('History')

print(len(my_courses))
```

Length of a list

How to find the length of a list? …by using **len()**

```
my_courses= ['mathematics', 'sciences', 'History']

print(len(my_courses))

my_courses.remove('History')

print(len(my_courses))
```

```
3
2
```

Sorting lists

```
print(my_courses)

my_courses.sort()

print(my_courses)
```

```
['mathematics', 'sciences', 'History']
['History', 'mathematics', 'sciences']
```

It sorts the list in alphabetical order, if the elements are numbers=> sort them from small to large….

```
my_courses.sort(reverse=True)

print(my_courses)
```

```
['sciences', 'mathematics', 'History']
```

It sorts the list in a decreasing order

All elements of a list have to have the same type?

All elements of a list have to have the same type?

No.

All elements of a list have to have the same type?

No.

Unlike some other programming languages, Python allows you to create lists with elements of different data types. This flexibility is one of the features that make Python lists versatile and easy to use.

```
mixed_list = [1, "hello", 6.3, [3, 4, 5]]
```

⚠ Attention: even though it is possible to use a list with different types, but use it wisely because it may be confusing and may complicate the work

⚠ Attention: It is not possible to sort this list

Example

1. Create a list of your favorite fruits.
2. Add one more fruit to the list.
3. Print the length of the list.
4. Remove one fruit from the list.
5. Sort the list in alphabetical order.

Example

1. Create a list of your favorite fruits.
2. Add one more fruit to the list.
3. Print the length of the list.
4. Remove one fruit from the list.
5. Sort the list in alphabetical order.

```
#Create a list of your favorite fruits.
fruits = ["banana", "apple", "cherry"]
```

```
#Add one more fruit to the list.
fruits.append("watermelon")
print(fruits)
```

```
['banana', 'apple', 'cherry', 'watermelon']
```

```
#Print the length of the list.
print(len(fruits))
```

```
4
```

```
#Remove one fruit from the list.
fruits.remove( "cherry")
print(fruits)
```

```
['banana', 'apple', 'watermelon']
```

```
#Sort the list in alphabetical order.
fruits.sort()
print(fruits)
```

```
['apple', 'banana', 'watermelon']
```

“For” loop

A «for» loop is used to iterate over an iterable object.

It executes a block of code repeatedly, once for each item in the iterable.

Examples of iterable objects in Python include lists, tuples, dictionaries, sets, and strings.

The syntax is:

```
for element in sequence:

    #code to be executed for each element
```

The loop keeps iterating until the end of the sequence is reached.

“For” loop: example

```
fruits = ["apple", "banana", "cherry", "watermelon"]
for f in fruits:
    print(f)
```

```
apple
banana
cherry
watermelon
```

For loop: range() function

What if we want to iterate over a list of numbers from 1 to 1000?

Shall we write : numbers=[1,2,3,……..] till 1000?

Of course not!

We can use the **range()** function

It is used to generate a sequence of numbers.

It can take one, two, or three arguments and returns a sequence of numbers based on those arguments.

For loop: range() function

range(*stop*)

range(*start*, *stop*)

range(*start*, *stop*, *step*)

- Start: optional. The starting value of the sequence. If omitted, it starts from 0.
- Stop: The sequence will be up to, but not including, this value.
- Step: optional. The interval between each number in the sequence. If omitted, the default step is 1.

```
for i in range(5):
    print(i)
```

```
for i in range(1,5):
    print(i)
```

```
for i in range(1,5,2):
    print(i)
```

For loop: range() function

range(*stop*)

range(*start*, *stop*)

range(*start*, *stop*, *step*)

- Start: optional. The starting value of the sequence. If omitted, it starts from 0.
- Stop: The sequence will be up to, but not including, this value.
- Step: optional. The interval between each number in the sequence. If omitted, the default step is 1.

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

```
for i in range(1,5):
    print(i)
```

```
1
2
3
4
```

```
for i in range(1,5,2):
    print(i)
```

```
1
3
```

For loop v.s while loop

Can you tell what’s the difference between the while loop and the for loop?

What is better?

For loop v.s while loop

While	For
You use it when you want to repeat a block of code until a certain condition is met, and the number of iterations is not known in advance.	You use it when you know the number of iterations or when iterating over a sequence
While loops can potentially lead to infinite loops if the termination condition is not met, so it's crucial to ensure that the condition eventually becomes “False”	For loops have a clear control flow based on the sequence length.

In summary, **for** loops are ideal for iterating over a known sequence. **while** loops are suitable when you need to repeatedly execute code based on a condition.
The choice between them depends on the specific requirements of your program.

Exercise 2: Multiplication Table

Ask the user to enter a number.

Print the multiplication table of this number.

Use a “for loop” to iterate from 1 to 10 to calculate and print the multiples.

Exercise 2: Multiplication Table

Ask the user to enter a number.

Print the multiplication table of this number.

Use a “for loop” to iterate from 1 to 10 to calculate and print the multiples.

```python
# Exercise 1: Multiplication Table

# Get input from the user
num = int(input("Enter a number: "))

# Print the multiplication table
for i in range(1, 11):
    result = num * i
    print(num, "x", i, "=", result)
```

```
Enter a number:  6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

Exercise 3: Factorial Calculation

Calculate the factorial of a number entered by the user using a for loop.

The factorial of a number n! is the product of all positive integers from 1 to n.

Exercise 3: Factorial Calculation

Calculate the factorial of a number entered by the user using a for loop.

The factorial of a number n! is the product of all positive integers from 1 to n.

```
# Get input from the user
num = int(input("Enter a number: "))

# Initialize the factorial to 1
factorial = 1

# Calculate the factorial using a for loop
for i in range(1, num + 1):
    factorial *= i

# Print the factorial
print("Factorial of ",num, " is: ",factorial)
```

```
Enter a number:  5
Factorial of  5  is:  120
```

Exercise 4: Sum of Numbers

Calculate the sum of all numbers from 1 to 100 (inclusive) using a for loop.

Exercise 5: Sum, product and average

Given the list [10,20,30,40,50,60,70,80,90], find:

- The sum of all the items
- The product result of all the items
- The average of all the items

Excercise 4:Sum of Numbers

```
sum_of_numbers = 0

# Calculate the sum of numbers from 1 to 100 using a for loop
for number in range(1, 101):
    sum_of_numbers += number

print("Sum of numbers from 1 to 100:", sum_of_numbers)
```

Exercise 5: Sum, product and average

```
# Given list
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Sum of all items
total_sum = sum(numbers)

# Product of all items
product = 1
for num in numbers:
    product *= num

# Average of all items
average = total_sum / len(numbers)

# Display results
print("Sum:", total_sum)
print("Product:", product)
print("Average:", average)
```

Exercise 6:

Write a Python program that repeatedly asks the user to enter 5 numbers (integers).

The program should:

- Use a while loop to keep asking until 5 valid integers are entered.
- Use a try-except block to handle invalid input (e.g., if the user types letters).

After collecting the numbers, print the list of entered numbers, and use a for loop to:

- Calculate and print the sum, maximum, and minimum values.
- Print each number and whether it’s even or odd.

Exercise 6: Input and Analyze a List of Numbers

```python
numbers = []

# Step 1: Collect 5 valid integers using while + try/except
while len(numbers) < 5:
    try:
        num = int(input(f"Enter number {len(numbers) + 1} : "))
        numbers.append(num)
    except ValueError:
        print("Invalid input! Please enter an integer.")

print("You entered:", numbers)
# Step 2: Use a for loop to analyze the numbers
print("Number analysis:")

for n in numbers:
    if n % 2 == 0:
        print(n, " is even")
    else:
        print(n, " is odd")

# Step 3: Compute and display results
total_sum = sum(numbers)
maximum = max(numbers)
minimum = min(numbers)

print("Sum = ", total_sum)
print("Maximum = ", maximum)
print("Minimum = ", minimum)
```